

הסבר מלא על הקוד

Redact-FileContent.ps1

מסמך זה עובר על כל שורה ושורה של הסקריפט, לפי סדר הופעתן בקובץ, ומסביר בעברית מה כל בלוק קוד עושה בפועל וכיצד הוא משתלב בזרימה הכללית של ההרצה - מקבלת הפרמטרים, דרך הדפסת השורות שהשתנו וספירת ההתאמות, ועד דיווח הנתיבים שאין אליהם הרשאת גישה.

316 שורות קוד

18 סעיפים

branch: claude/master

מבנה הסקריפט לפי הסדר

lines 1-62	1	כותרת התיעוד הפנימי (Comment-Based Help)
lines 64-107	2	הגדרת הפרמטרים של הסקריפט (param block)
lines 109-112	3	בדיקת תקינות הנתיב
lines 114-116	4	הכנת תיקיית הגיבוי (רק אם -Backup פעיל)
lines 118-129	5	איסוף כל הקבצים ואיסוף שגיאות הגישה
lines 131-154	6	מבנה הנתונים למיפוי עקבי + הפונקציה Get-PlaceholderName
lines 156-169	7	המונים הגלובליים והפונקציה Write-ChangedLines
lines 171-186	8	תחילת הלולאה הראשית - מעבר על כל קובץ
lines 188-194	9	מעבר שורה-שורה: החלפות מילוליות (\$Replacements)
lines 196-199	10	מחיקת תיאור פורט (-StripPortDescriptions)
lines 201-223	11	שלושת שלבי האנונימיזציה בתוך הלולאה
lines 225-238	12	שמירת השורה שהשתנתה וספירת ההתאמות בה
lines 240-256	13	סיום הלולאה הפנימית וההחלטה אם לכתוב לקובץ
lines 258-266	14	ביצוע הגיבוי בפועל
lines 268-271	15	כתיבת הקובץ המעודכן והדפסת השורות שהשתנו
lines 273-283	16	הדפסת סיכום הרצה כללי + סך ההתאמות
lines 285-299	17	הדפסת טבלאות המיפוי (מה הוחלף במה)
lines 301-316	18	דוח הנתיבים שאין אליהם הרשאת גישה

1 כותרת התיעוד הפנימי (Comment-Based Help)

```
1 <#
2 Redact-FileContent.ps1
3
4 Unified replacement/redaction engine for files, recursively through subfolders.
5 All modes below share the same file-processing, dry-run and backup logic.
6
7 1. Literal/regex replacements ($Replacements below) - always applied.
8 Each key is a regex pattern, each value is what to replace it with.
9 (Companion to Search-FilesForWord.ps1 - use that one first to preview matches.)
10
11 2. -AnonymizeSwitchNames - finds "S"/"s" followed by letters/digits/special
12 characters and ending in "nn" (e.g. S-DC1-01nn), anywhere in a line - including
13 glued to a prefix like "Uplink_to_S-DC1-01nn" - and replaces just that part
14 with a sequential generic placeholder (SWITCH001, SWITCH002, ...).
15
16 3. -AnonymizeUsernames - finds "user" followed directly by digits (e.g.
17 "user22351"), anywhere in a line - including glued to other text like
18 "PC_user22351" or "user10023_workstation" - and replaces just that part with
```

```

19         a sequential placeholder (USER001, USER002, ...).
20
21     4. -AnonymizeIPs - finds IPv4 addresses and replaces each with a sequential
22        placeholder (IP001, IP002, ...).
23
24     5. -StripPortDescriptions - removes the description text from interface lines
25        such as "GigabitEthernet1/0/1 - TO_SWITCH001", leaving just the port name
26        ("GigabitEthernet1/0/1"). Covers GigabitEthernet, TenGigabitEthernet,
27        TwentyFiveGigabitEthernet, FastEthernet and Ethernet ports.
28
29     For modes 2-4, the same original value always maps to the same placeholder
30     within a single run (each category has its own counter), but nothing is
31     persisted between runs - running the script again assigns fresh numbers.
32
33     All write modes support -DryRun (report what would change, touch nothing) and
34     -Backup (copy each modified file's original content aside before overwriting it).
35
36     REPORTING:
37
38     * Every changed line is printed under the file it belongs to, with its line
39       number, how many matches it contained, and the line before ("-") and after
40       ("+") the replacement.
41
42     * Each file reports its match count, and the run ends with the total number of
43       matches and changed lines across all files.
44
45     * Folders and files that could not be opened (typically "access denied") are
46       not skipped silently - they are listed at the end with their path and reason.
47
48     USAGE EXAMPLES:
49
50     # Preview the literal replacements only, no files modified
51     .\Redact-FileContent.ps1 -Path "D:\\" -DryRun
52
53     # Apply literal replacements for real, keeping backups
54     .\Redact-FileContent.ps1 -Path "D:\\" -Backup
55
56     # Anonymize switch names, usernames and IPs, and strip port descriptions
57     .\Redact-FileContent.ps1 -Path "D:\\" -AnonymizeSwitchNames -AnonymizeUsernames -
58     AnonymizeIPs -StripPortDescriptions -Backup
59
60     # Anonymization only - skip the literal $Replacements table
61     .\Redact-FileContent.ps1 -Path "D:\\" -AnonymizeIPs -Replacements @{} -DryRun
62
63     # Limit to specific file types
64     .\Redact-FileContent.ps1 -Path "D:\Logs" -Include *.log,*.txt -AnonymizeUsernames
65     s -DryRun
66     #>

```

הבלוק בין <# ל-#> הוא הערת תיעוד (block comment) שלא מתבצעת בזמן ריצה. היא מסבירה למי שפותח את הקובץ, לפני שהוא בכלל מריץ אותו, מה הסקריפט עושה:

- הסקריפט הוא "מנוע החלפה/מחיקת מידע" (Redaction) (אחד שרץ רקורסיבית על כל תיקיית משנה תחת נתיב נתון).
- קיימים 5 סוגי פעולות שאפשר להפעיל, כל אחת בנפרד או כמה יחד: החלפות מילוליות (תמיד פעיל), אנונימיזציה של שמות מתגים, אנונימיזציה של שמות משתמשים, אנונימיזציה של כתובות IP, ומחיקת תיאורי פורטים.
- הפסקה **REPORTING** (שורות 36-44) היא החדשה: היא מתעדת שכל שורה שהשתנתה מודפסת תחת הקובץ שלה, שנספרות כמות ההתאמות לכל קובץ ובסך הכל, ושתיקיות שאין אליהן הרשאת גישה מדווחות בסוף ולא נבלעות בשקט.
- בסוף מופיעות דוגמאות שימוש בפועל - איך מריצים את הסקריפט משורת הפקודה עם צירופי פרמטרים שונים.

```

64 [CmdletBinding(SupportsShouldProcess = $true)]
65 param(
66     # Root folder to process. Defaults to the current directory.
67     [string]$Path = ".",
68
69     # Optional file name filters (wildcards), e.g. *.txt, *.log, *.csv.
70     # Leave empty to process every file.
71     [string[]]$Include = @("*"),
72
73     # Ordered map of regex pattern -> replacement text, applied on every line.
74     [System.Collections.Specialized.OrderedDictionary]$Replacements = [ordered]@{
75         "ass" = "aaa"
76     },
77
78     # Anonymize switch/device name tokens matching -SwitchNamePattern.
79     [switch]$AnonymizeSwitchNames,
80     [string]$SwitchNamePattern = '[Ss][^\s]*nn(?:!\S)',
81     [string]$SwitchNamePrefix = "SWITCH",
82
83     # Anonymize username tokens matching -UsernamePattern.
84     [switch]$AnonymizeUsernames,
85     [string]$UsernamePattern = 'user\d+',
86     [string]$UsernamePrefix = "USER",
87
88     # Anonymize IPv4 addresses.
89     [switch]$AnonymizeIPs,
90     [string]$IPPattern = '(?<!\d)(?: (?:(?:25[0-5]|2[0-4]\d|1?\d?\d)\.){3}(?:25[0-5]|2[0-4]\d|1?\d?\d)(?!\d))',
91     [string]$IPPrefix = "IP",
92
93     # Strip the description text off interface lines, e.g.
94     # "GigabitEthernet1/0/1 - TO_SWITCH001" -> "GigabitEthernet1/0/1".
95     [switch]$StripPortDescriptions,
96     [string]$PortDescriptionPattern = '^\\s*(?<port>(?:TwentyFiveGigabitEthernet|TenGigabitEthernet|GigabitEthernet|FastEthernet|Ethernet)\\d+(?:/\\d+)*\\s*-\\s*.*$',
97
98     # Report matches and what would change, without modifying any file.
99     [switch]$DryRun,
100
101     # Copy each file's original content into -BackupFolder before overwriting it.
102     [switch]$Backup,
103
104     # Folder to hold backups when -Backup is used. Defaults to a timestamped
105     # folder created next to -Path. Relative structure under -Path is preserved.
106     [string]$BackupFolder
107 )

```

זוהי ה"חתימה" של הסקריפט - כל הפרמטרים שאפשר להעביר לו כשמריצים אותו. שורה 64 - `[CmdletBinding(SupportsShouldProcess = $true)]` - הופכת את הסקריפט ל-cmdlet "אמיתי" של PowerShell, ובפרט מפעילה תמיכה ב- `-WhatIf` וב- `-Confirm`.

- `-Path` - תיקיית השורש שעליה רצים. ברירת מחדל: התיקיה הנוכחית.
- `-Include` - מסוננת קבצים לפי שם/סיומת (למשל `*.txt`). ברירת מחדל: הכל.
- `-Replacements` - טבלה מסודרת שבה כל מפתח הוא תבנית regex וכל ערך הוא הטקסט שיחליף אותה. פעילה תמיד.
- `-AnonymizeSwitchNames` / `-SwitchNamePattern` / `-SwitchNamePrefix` - הדגל שמפעיל אנונימיזציה

שמות מתגים, ה-regex שמזהה אותם, והקידומת לשם הגנרי.

- `-AnonymizeUsernames` / `-UsernamePattern` / `-UsernamePrefix` - אותו רעיון עבור שמות משתמשים (`userNNNN`).
- `-AnonymizeIPs` / `-IPPattern` / `-IPPrefix` - אותו רעיון עבור כתובות IPv4.
- `-StripPortDescriptions` / `-PortDescriptionPattern` - דגל שמפעיל מחיקת תיאורי פורטים, וה-regex שמזהה שורת פורט.
- `-DryRun` - מצב "הרצה יבשה": רק מדווח מה היה משתנה, לא נוגע באף קובץ.
- `-Backup` / `-BackupFolder` - האם לגבות כל קובץ שמשתנה לפני הכתיבה, ולאן.

3 בדיקת תקינות הנתיב

```
109 if (-not (Test-Path -LiteralPath $Path)) {  
110     Write-Error "Path not found: $Path"  
111     return  
112 }
```

לפני שעושים כלום, בודקים עם `Test-Path` שהנתיב שהוזן ב- `-Path` באמת קיים על הדיסק. אם לא - מודפסת שגיאה עם `Write-Error` והפקודה `return` עוצרת את כל הסקריפט מיידית, כדי שלא ננסה להמשיך לעבוד על נתיב שלא קיים.

4 הכנת תיקיית הגיבוי (רק אם Backup פעיל)

```
114 if ($Backup -and -not $BackupFolder) {  
115     $BackupFolder = Join-Path (Resolve-Path -LiteralPath $Path) "_backup_$(Get-Date -For  
116     mat 'yyyyMMdd_HH:mm:ss')"  
117 }
```

אם המשתמש ביקש `-Backup` אבל לא ציין באופן מפורש `-BackupFolder`, הסקריפט בונה שם תיקייה אוטומטי: תיקייה חדשה בשם `_backup_` ואחריה תאריך ושעה מדויקים (עד השנייה), שנוצרת ליד הנתיב המקורי. כך כל הרצה עם גיבוי מקבלת תיקיית גיבוי משלה, ולא דורסת גיבוי קודם.

5 איסוף כל הקבצים ואיסוף שגיאות הגישה

```
118 $resolvedRoot = (Resolve-Path -LiteralPath $Path).ProviderPath  
119  
120 # Enumeration errors are collected instead of being swallowed, so folders that  
121 # could not be opened (typically "access denied") are reported at the end  
122 # rather than being skipped silently.  
123 $allFiles = Get-ChildItem -Path $Path -Include $Include -Recurse -File -ErrorAction SilentlyContinue -ErrorVariable accessErrors  
124  
125 $ignoreCase = [System.Text.RegularExpressions.RegexOptions]::IgnoreCase  
126 $switchNameRegex = [regex]::new($SwitchNamePattern, $ignoreCase)  
127 $usernameRegex = [regex]::new($UsernamePattern, $ignoreCase)  
128 $ipRegex = [regex]::new($IPPattern)
```

```
129 $portDescriptionRegex = [regex]::new($PortDescriptionPattern, $ignoreCase)
```

- שורה 118: `$resolvedRoot` הופך את `-Path` לנתיב מלא ומוחלט. זה נדרש כדי לחשב בהמשך את הנתיב היחסי של כל קובץ בתוך תיקיית הגיבוי.
- שורה 123 - זו השורה ששונתה: `Get-ChildItem` אוסף את כל הקבצים תחת `-Path` רקורסיבית בעזרת `-Recurse`, עם הסינון של `-Include`. הדגל `-File` מוציא תיקיות מהתוצאה.
- בעבר `ErrorAction SilentlyContinue` לבדו גרם לסקריפט "לדלג בשקט" על תיקיות שאין הרשאה לגשת אליהן - כלומר הן פשוט נעלמו מהסריקה בלי שום סימן. עכשיו נוסף `-ErrorVariable accessErrors`: הריצה עדיין לא נעצרת, אבל כל שגיאה נאספת למשתנה `$accessErrors` במקום להימחק, ובסוף ההרצה (סעיף 18) מודפסת ממנה רשימת הנתיבים שנחסמו.
- שורות 125-129: בונים מראש ארבעה אובייקטי `[regex]` מקומפלים - אחד לכל תבנית: מתגים, משתמשים, IP ותיאורי פורטים - כדי לא לקמפל מחדש את אותו `regex` בכל שורה בכל קובץ. כולם חוץ מזה של IP מקבלים את הדגל `IgnoreCase`.

6 מבנה הנתונים למיפוי עקבי + הפונקציה `Get-PlaceholderName`

```
131 # Original value (as first encountered) -> generic placeholder, one map per category.
132 # Consistent for this run only - not persisted between runs.
133 $switchNameMap = [ordered]@{}
134 $usernameMap = [ordered]@{}
135 $ipMap = [ordered]@{}
136 $switchNameIndex = 1
137 $usernameIndex = 1
138 $ipIndex = 1
139
140 function Get-PlaceholderName {
141     param(
142         [string]$OriginalValue,
143         [System.Collections.Specialized.OrderedDictionary]$Map,
144         [string]$Prefix,
145         [string]$IndexVarName
146     )
147
148     if (-not $Map.Contains($OriginalValue)) {
149         $index = Get-Variable -Name $IndexVarName -Scope Script -ValueOnly
150         $Map[$OriginalValue] = "{0}{1:D3}" -f $Prefix, $index
151         Set-Variable -Name $IndexVarName -Scope Script -Value ($index + 1)
152     }
153     return $Map[$OriginalValue]
154 }
```

זהו ה"לב" של מנגנון האנונימיזציה העקבית. לכל אחת משלוש הקטגוריות (מתגים/משתמשים/IP) יש:

- מפה (Ordered Dictionary) - `$switchNameMap`, `$usernameMap`, `$ipMap` - ששומרת לכל ערך מקורי את השם הגנרי שהוקצה לו, לאורך כל ההרצה (על פני כל הקבצים, לא רק קובץ אחד).
- מונה (Index) - `$switchNameIndex` וכו' - כל אחד מתחיל מ-1, ועולה בכל פעם שנתקלים בערך חדש באותה קטגוריה.

הפונקציה `Get-PlaceholderName` (שורות 140-154) מקבלת ערך מקורי, את המפה המתאימה, קידומת (כמו "SWITCH") ואת שם המשתנה של המונה. אם הערך עדיין לא קיים במפה - נבנה ממנו שם חדש בפורמט

SWITCH001 והמונה עולה ב-1; אם הוא כבר קיים - מוחזר אותו שם גנרי שהוקצה לו קודם, כך שאותו **S-DC1-01nn** תמיד יקבל את אותו **SWITCH001** בכל מקום שהוא מופיע.

7 המונים הגלובליים והפונקציה Write-ChangedLines

```
156 $changedCount = 0
157 $scannedCount = 0
158 $totalMatchCount = 0
159 $totalChangedLines = 0
160
161 function Write-ChangedLines {
162     param([object[]]$Lines)
163
164     foreach ($changedLine in $Lines) {
165         Write-Host ("    line {0} ({1} match(es)):" -f $changedLine.Line, $changedLine.M
atches) -ForegroundColor DarkGray
166         Write-Host ("        - {0}" -f $changedLine.Before) -ForegroundColor DarkRed
167         Write-Host ("        + {0}" -f $changedLine.After) -ForegroundColor DarkGreen
168     }
169 }
```

כאן מרוכזים ארבעת המונים של ההרצה, ומיד אחריהם הפונקציה החדשה שמדפיסה את השורות שהשתנו:

- **\$changedCount** - כמה קבצים שונו בפועל; **\$scannedCount** - כמה קבצים נסרקו בסך הכל.
- **\$totalMatchCount** ו- **\$totalChangedLines** הם החדשים: סך כל ההתאמות שנמצאו בכל הקבצים, וסך כל השורות שהשתנו. הם מודפסים בסוף ההרצה (סעיף 16).
- **Write-ChangedLines** (שורות 161-169) מקבלת רשימה של שורות שהשתנו, ולכל אחת מדפיסה שלוש שורות פלט: מספר השורה וכמות ההתאמות בה, ואחריהן השורה לפני השינוי (מסומנת -), באדום (והשורה אחרי השינוי (מסומנת +, בירוק) - בדיוק כמו **diff**.

8 תחילת הלולאה הראשית - מעבר על כל קובץ

```
171 foreach ($fileItem in $allFiles) {
172     $scannedCount++
173
174     $content = Get-Content -LiteralPath $fileItem.FullName -ErrorAction SilentlyContinue
-ErrorVariable +accessErrors
175     if ($null -eq $content) {
176         continue
177     }
178
179     $fileMatchCount = 0
180     $fileChangedLines = [System.Collections.Generic.List[object]]::new()
181     $lineNumber = 0
182
183     $updatedContent = foreach ($row in $content) {
184         $lineNumber++
185         $currentRow = $row
186         $script:rowMatchCount = 0
```

הלולאה **foreach (\$fileItem in \$allFiles)** עוברת על כל קובץ שנאסף בסעיף 5. עבור כל קובץ:

- המונה **\$scannedCount** עולה ב-1.

- שורה 174: `Get-Content` קוראת את הקובץ כמערך שורות. גם כאן נוסף `-ErrorVariable +accessErrors` - הסימן `+` אומר "הוסף לרשימה הקיימת", כך שגם קובץ בודד שנעול או חסום להרשאות נאסף לאותו דוח בסוף, ולא רק תיקיות.
- אם הקריאה נכשלה (`$content` ריק/`continue` - null מדלג לקובץ הבא בלי לגעת בו).
- שורות 179-181 מאתחלים לכל קובץ שלושה משתנים: `$fileMatchCount` - מונה ההתאמות בקובץ; `$fileChangedLines` - רשימת השורות שהשתנו בקובץ, חדש; ו- `$LineNumber` - מונה מספר השורה בתוך הקובץ, חדש גם הוא, כדי שאפשר יהיה להדפיס לכל שינוי את מספר השורה שלו.
- שורה 186: `$script:rowMatchCount = 0` מאפס את מונה ההתאמות של השורה הנוכחית. זה מונה חדש ברמת השורה (ולא רק ברמת הקובץ), והוא ב-scope של הסקריפט כדי שגם ה-script-blocks של האנונימיזציה יוכלו להעלות אותו.

9 מעבר שורה-שורה: החלפות מילוליות \$(Replacements)

```
188     foreach ($search in $Replacements.Keys) {
189         $lineMatches = [regex]::Matches($currentRow, $search).Count
190         if ($lineMatches -gt 0) {
191             $script:rowMatchCount += $lineMatches
192             $currentRow = $currentRow -replace $search, $Replacements[$search]
193         }
194     }
```

עבור כל שורה בקובץ עוברים על כל תבנית ב- `$Replacements`. לכל תבנית: סופרים כמה פעמים היא מופיעה בשורה עם `[regex]::Matches(...).Count`, מוסיפים את המספר הזה למונה של השורה (`$script:rowMatchCount`, ורק אם יש לפחות התאמה אחת - מבצעים בפועל את ההחלפה עם האופרטור `-replace`).

10 מחיקת תיאור פורט \$(StripPortDescriptions)

```
196     if ($StripPortDescriptions -and $portDescriptionRegex.IsMatch($currentRow)) {
197         $script:rowMatchCount++
198         $currentRow = $portDescriptionRegex.Replace($currentRow, '${port}')
199     }
```

אם הדגל `StripPortDescriptions` פעיל, בודקים אם השורה כולה תואמת את `$PortDescriptionPattern` (למשל `GigabitEthernet1/0/1 - TO_SWITCH001`). אם כן - מעלים את מונה השורה ב-1, ומחליפים את כל השורה בקבוצת ה- `(?<port>...)` שנתפסה, כלומר משאירים רק את שם הפורט ומוחקים את המקף ואת הטקסט שאחריו.

השלב הזה מתבצע לפני שלושת שלבי האנונימיזציה, כך שאם גם `StripPortDescriptions` וגם `AnonymizeSwitchNames` פעילים - קודם נמחק ה-description, ורק מה שנשאר בשורה עובר הלאה לבדיקת האנונימיזציה.

11 שלושת שלבי האנונימיזציה בתוך הלולאה

```
201     if ($AnonymizeSwitchNames) {
202         $currentRow = $switchNameRegex.Replace($currentRow, {
203             param($match)
204             $script:rowMatchCount++
205             Get-PlaceholderName -OriginalValue $match.Value -Map $switchNameMap -Pre
fix $SwitchNamePrefix -IndexVarName "switchNameIndex"
206         })
207     }
208
209     if ($AnonymizeUsernames) {
210         $currentRow = $usernameRegex.Replace($currentRow, {
211             param($match)
212             $script:rowMatchCount++
213             Get-PlaceholderName -OriginalValue $match.Value -Map $usernameMap -Prefi
x $UsernamePrefix -IndexVarName "usernameIndex"
214         })
215     }
216
217     if ($AnonymizeIPs) {
218         $currentRow = $ipRegex.Replace($currentRow, {
219             param($match)
220             $script:rowMatchCount++
221             Get-PlaceholderName -OriginalValue $match.Value -Map $ipMap -Prefix $IPP
refix -IndexVarName "ipIndex"
222         })
223     }
```

שלושה בלוקים כמעט זהים במבנה - אחד לכל קטגוריה (מתגים, משתמשים, IP). כל בלוק פעיל רק אם הדגל המתאים דלוק. בכל בלוק:

- קוראים ל- `$xxxRegex.Replace($currentRow, {...})` - הפעולה `.Replace`. מקבלת `script-block` `MatchEvaluator` (במקום מחרוזת קבועה, כלומר לכל התאמה מריצים פונקציה שמחליטה מה יהיה הטקסט המחליף).
- בתוך אותו `script-block`, `$script:rowMatchCount++` מעלה עכשיו את מונה ההתאמות של השורה (בעבר זה היה מונה הקובץ), ואז `Get-PlaceholderName` מחזירה את השם הגנרי העקבי שיחליף את הטקסט המקורי.
- שלושת השלבים רצים לפי הסדר: קודם שמות מתגים, אחר כך שמות משתמשים, ולבסוף כתובות IP - כל אחד פועל על התוצאה של הקודם לו באותה שורה.

12 שמירת השורה שהשתנתה וספירת ההתאמות בה

```
225     # Keep the line itself, so every change can be printed with its line
226     # number and how many matches it contained.
227     if ($script:rowMatchCount -gt 0) {
228         $fileMatchCount += $script:rowMatchCount
229         $fileChangedLines.Add([PSCustomObject]@{
230             Line      = $lineNumber
231             Matches    = $script:rowMatchCount
232             Before     = $row.Trim()
233             After      = $currentRow.Trim()
234         })
235     }
```

```

235     }
236
237     $currentRow
238 }

```

זהו הבלוק החדש שמאפשר להדפיס בפועל את השורות. אחרי שכל ההחלפות רצו על השורה, בודקים אם `$script:rowMatchCount` גדול מאפס - כלומר האם השורה הזו בכלל השתנתה:

- אם כן, מוסיפים את מונה השורה למונה הקובץ (`$fileMatchCount += $script:rowMatchCount`).
- ואז שומרים ב- `$fileChangedLines` אובייקט עם ארבעה שדות: `Line` - מספר השורה בקובץ; `Matches` - כמה התאמות היו בשורה; `Before` - השורה המקורית; ו- `After` - השורה אחרי ההחלפה. זה מה שמאפשר להדפיס אחר כך את השורה עצמה, ולא רק את שם הקובץ.
- שורה 237: `$currentRow` מוחזרת מהלולאה הפנימית ונאספת ל- `$updatedContent` - כך שבסוף יש לנו את כל הקובץ מחדש, שורה אחר שורה, עם כל השינויים.

13 סיום הלולאה הפנימית וההחלטה אם לכתוב לקובץ

```

240     if ($fileMatchCount -eq 0) {
241         continue
242     }
243
244     $changedCount++
245     $totalMatchCount += $fileMatchCount
246     $totalChangedLines += $fileChangedLines.Count
247
248     if ($DryRun) {
249         Write-Host "[DryRun] Would update: $($fileItem.FullName) ($fileMatchCount match(
250 es) on $($fileChangedLines.Count) line(s))" -ForegroundColor DarkYellow
251         Write-ChangedLines -Lines $fileChangedLines
252         continue
253     }
254     if (-not $PSCmdlet.ShouldProcess($fileItem.FullName, "Replace $fileMatchCount match(
255 es)")) {
256         continue
257     }

```

- שורות 240-242: אם אף שורה בקובץ לא השתנתה (`$fileMatchCount -eq 0`) - מדלגים לקובץ הבא. הקובץ לא נכתב מחדש, לא מגובה ולא נספר כ"שונה".
- שורות 244-246: אחרת, מונה הקבצים ששונו עולה ב-1, ובנוסף מתעדכנים שני המונים הכלליים החדשים - `$totalMatchCount` (סך ההתאמות) ו- `$totalChangedLines` (סך השורות שהשתנו).
- שורות 248-252: במצב `-DryRun` מודפסת שורת דיווח כתומה עם שם הקובץ, מספר ההתאמות ומספר השורות שהשתנו, ומיד אחריה `Write-ChangedLines` מדפיסה את כל השורות עצמן (לפני/אחרי) - כך שאפשר לראות בדיוק מה היה משתנה, בלי לגעת באף קובץ.
- שורות 254-256: `$PSCmdlet.ShouldProcess(...)` הוא המנגנון הפנימי של PowerShell שמפעיל את `-WhatIf / -Confirm`. אם המשתמש הריץ עם `-WhatIf`, השורה הזו תחזיר `$false` והקובץ לא ייכתב - בלי צורך בקוד נוסף.

```

258     if ($Backup) {
259         $relativePath = $fileItem.FullName.Substring($resolvedRoot.Length).TrimStart('\', '/')
260         $backupPath = Join-Path $BackupFolder $relativePath
261         $backupDir = Split-Path -Path $backupPath -Parent
262         if (-not (Test-Path -LiteralPath $backupDir)) {
263             New-Item -ItemType Directory -Path $backupDir -Force | Out-Null
264         }
265         Copy-Item -LiteralPath $fileItem.FullName -Destination $backupPath -Force
266     }

```

אם `-Backup` פעיל: מחשבים את הנתיב היחסי של הקובץ ביחס לתיקיית השורש (`$resolvedRoot` שחושבה בשורה 118), ובונים ממנו נתיב מקביל בתוך תיקיית הגיבוי - כך שמבנה התיקיות המקורי נשמר גם בגיבוי. אם תיקיית היעד עדיין לא קיימת, `New-Item -ItemType Directory` יוצרת אותה כולל כל תיקיות הביניים (החסרות) בזכות `-Force`. לבסוף `Copy-Item` מעתיקה את הקובץ המקורי, לפני השינוי, למיקום הגיבוי.

15 כתיבת הקובץ המעודכן והדפסת השורות שהשתנו

```

268     Write-Host "Working on: $($fileItem.FullName) ($fileMatchCount match(es) on $($fileC
hangedLines.Count) line(s))" -ForegroundColor DarkYellow
269     Write-ChangedLines -Lines $fileChangedLines
270     $updatedContent | Set-Content -LiteralPath $fileItem.FullName
271 }

```

שורה 268 מדפיסה שורת סטטוס צהובה עם שם הקובץ, מספר ההתאמות שנמצאו בו וכמה שורות השתנו. שורה 269 היא החדשה: `Write-ChangedLines` מדפיסה מיד אחריה את כל השורות של אותו קובץ - לכל אחת מספר שורה, כמות התאמות, והשורה לפני ואחרי. שורה 270, `Set-Content`, כותבת את `$updatedContent` בחזרה לאותו קובץ. זהו הרגע היחיד שבו קובץ באמת משתנה על הדיסק.

16 הדפסת סיכום הרצה כללי + סך ההתאמות

```

273 Write-Host ""
274 if ($DryRun) {
275     Write-Host "Dry run complete. $changedCount of $scannedCount file(s) would be update
d." -ForegroundColor Green
276 } else {
277     Write-Host "Complete successfully! $changedCount of $scannedCount file(s) updated."
-ForegroundColor Green
278     if ($Backup) {
279         Write-Host "Backups saved under: $BackupFolder" -ForegroundColor Green
280     }
281 }
282
283 Write-Host "Total matches found: $totalMatchCount on $totalChangedLines line(s)." -Foreg
roundColor Green

```

אחרי שהלולאה על כל הקבצים הסתיימה, מודפס סיכום: במצב `-DryRun` - כמה קבצים היו משתנים מתוך כמה שנסרקו; אחרת - כמה קבצים אכן שונו, ואם היה גיבוי גם הנתיב המלא לתיקיית הגיבוי.

שורה 283 היא החדשה: `Total matches found: N on M line(s)` - סך כל ההתאמות שנמצאו בכל הקבצים יחד, וסך כל השורות שהשתנו. זו התשובה לשאלה "כמה פעמים המילה נמצאה" ברמת ההרצה כולה, בנוסף לספירה שמודפסת לכל קובץ בנפרד.

17 הדפסת טבלאות המיפוי (מה הוחלף במה)

```
285 function Write-NameMapping {
286     param([string]$Title, [System.Collections.Specialized.OrderedDictionary]$Map)
287
288     if ($Map.Count -eq 0) {
289         return
290     }
291     Write-Host "`n$Title mapping for this run:" -ForegroundColor Cyan
292     $Map.GetEnumerator() | ForEach-Object {
293         Write-Host (" {0} -> {1}" -f $_.Key, $_.Value)
294     }
295 }
296
297 if ($AnonymizeSwitchNames) { Write-NameMapping -Title "Switch name" -Map $switchNameMap }
298 if ($AnonymizeUsernames) { Write-NameMapping -Title "Username" -Map $usernameMap }
299 if ($AnonymizeIPs) { Write-NameMapping -Title "IP address" -Map $ipMap }
```

הפונקציה `Write-NameMapping` (שורות 285-295) מקבלת כותרת ומפה, ואם יש בה לפחות ערך אחד - מדפיסה כותרת כחולה ואז שורה לכל ערך מקורי עם השם הגנרי שהוקצה לו (למשל `S-DC1-01nn -> SWITCH001`). שלוש השורות 297-299 קוראות לפונקציה פעם אחת לכל קטגוריה שהייתה פעילה, כדי שבסוף ההרצה יהיה למפעיל תיעוד מלא של מה הוחלף במה - גם אם המיפוי עצמו לא נשמר לקובץ ולא יישמש בהרצה הבאה.

18 דוח הנתונים שאין אליהם הרשאת גישה

```
301 # Folders and files that could not be opened, with the reason. TargetObject holds
302 # the path for these provider errors; fall back to the error's target name.
303 $inaccessible = $accessErrors |
304     ForEach-Object {
305         [PSCustomObject]@{
306             Path = if ($_.TargetObject) { $_.TargetObject } else { $_.CategoryInfo.Tar
307             getName }
308             Reason = $_.Exception.Message
309         }
310     } |
311     Where-Object { $_.Path } |
312     Sort-Object Path -Unique
313
314 if ($inaccessible) {
315     Write-Host "`nPaths that could not be accessed and were skipped: $($inaccessible).
316     Count)" -ForegroundColor Yellow
317     $inaccessible | Format-Table -AutoSize -Wrap
318 }
```

זהו הסעיף החדש שסוגר את הפער: כל השגיאות שנאספו ל- `$accessErrors` גם מ- `Get-ChildItem` בשורה 123 וגם מ- `Get-Content` בשורה 174 (מומרות כאן לרשימה קריאה).

- לכל שגיאה בונים אובייקט עם שני שדות: `Path` - הנתוב שנחסם, ו- `Reason` - הודעת השגיאה עצמה (בדרך

כלל Access to the path ... is denied).

- בשגיאות מסוג זה הנתיב יושב ב- `$_TargetObject`; אם משום מה הוא ריק, נופלים חזרה ל- `$_CategoryInfo.TargetName`.
- `where-Object { $_.Path }` מסנן שגיאות בלי נתיב, ו- `Sort-Object Path -Unique` ממין ומוריד כפילויות, כדי שאותה תיקייה לא תופיע פעמיים.
- אם נשאר משהו ברשימה, מודפסת כותרת צהובה עם מספר הנתיבים שנחסמו ואחריה טבלה של נתיב + סיבה. כך ברור בסוף כל הרצה אם הסריקה הייתה מלאה, או שהיו תיקיות שהסקריפט לא הצליח להיכנס אליהן.